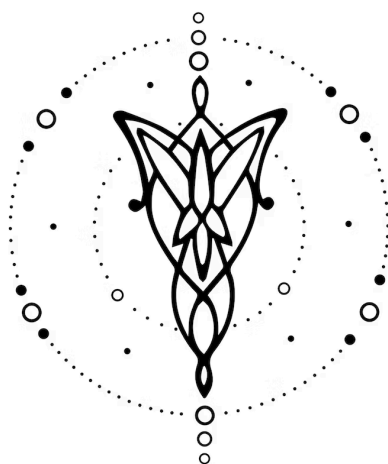




Especificación – Tax as Code

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

2 de Abril de 2026

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Santiago	Cibeira	64560	scibeira@itba.edu.ar
Manuel	Suñol	64138	msunol@itba.edu.ar
Augusto	Ospal	63669	aospalmadeo@itba.edu.ar

2. Repositorio

La solución será versionada en: <https://github.com/manuel-sunol/tla-tp>.

3. Dominio

El lenguaje a desarrollar es un DSL (Domain Specific Language) denominado Tax-as-Code (TaC). El dominio seleccionado abarca la configuración, automatización y aplicación de reglas de facturación, retenciones impositivas, pagos a proveedores y cierres contables en el ecosistema corporativo argentino. La necesidad arquitectural surge de la fricción operativa que sufren empresas como el conglomerado ELVEN DOOR al lidiar con regímenes tributarios cambiantes, padrones mensuales y auditorías de riesgo legal y fiscal.

El lenguaje utiliza un paradigma declarativo basado en estructuras jerárquicas por indentación (estilo viñetas), ocultando la complejidad algorítmica y técnica para favorecer la Experiencia de Usuario (DX) de los profesionales contables. El backend del compilador procesará estas reglas para generar programas en lenguajes de propósito general (como C o Python) que actuarán como motores de reglas. Estos artefactos de salida podrán ejecutar cron jobs para sincronizar datos impositivos, auditar riesgos en tiempo real al facturar/pagar, y generar reportes duales (TXT y Excel) para la liquidación de impuestos.

4. Construcciones

El lenguaje TaC ofrecerá las siguientes construcciones y funcionalidades:

- (I). **Jerarquía por indentación:** El lenguaje no utiliza llaves de apertura y cierre, sino que define los scopes y bloques lógicos mediante tabulaciones y tokens de viñetas (`*` y `-`).
- (II). **Definición de Entidades (`cliente`, `proveedor`):** Construcciones para instanciar actores comerciales con sus atributos fiscales clave (tipo de persona, condición frente al IVA, jurisdicción).
- (III). **Reglas Operativas (`regla_facturacion`, `regla_pago`, `regla_exencion`):** Bloques que definen acciones a ejecutar al momento de emitir un comprobante o una orden de pago, permitiendo consultar bases de datos externas de forma abstracta y evaluar riesgos.
- (IV). **Automatización Asincrónica (`sincronizacion_padrones`, `cierre_mensual`):** Estructuras para programar tareas en segundo plano, como la descarga de padrones provinciales o la generación de reportes de auditoría.
- (V). **Tipos de datos:** Cadenas de texto (`texto`), numéricos, booleanos (`verdadero`, `falso`), porcentajes (ej. `21%`), identificadores de tiempo (`ultimo_dia_mes`), y fechas.
- (VI). **Operadores lógicos y relacionales:** Para validar vigencias de certificados o topes de facturación utilizando lenguaje natural en lugar de símbolos matemáticos (ej. `fecha_factura menor o igual a fecha_hasta`, `igual a`, `mayor a`).
- (VII). **Arreglos/Listas:** Utilizando corchetes (`[ ]`) para agrupar múltiples valores, parámetros o formatos de salida requeridos.
- (VIII). **Comentarios:** De línea simple utilizando `//`.

5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que instancie entidades **cliente** y **proveedor** con atributos básicos y booleanos.
- (II). Un programa que defina una **regla_facturacion** con consultas estáticas a bases externas (padrones).
- (III). Un programa que configure un bloque **sincronizacion_padrones** definiendo las frecuencias de ejecución.
- (IV). Un programa que asigne una lista de formatos de salida en un **cierre_mensual** usando corchetes `[]`.
- (V). Un programa que evalúe una condición relacional de fechas para dar de baja una exención impositiva.
- (VI). Un programa que utilice una **regla_pago** condicionada exclusivamente a proveedores con estado "Monotributista".
- (VII). Un programa que audite el riesgo laboral buscando parámetros históricos (**facturas_correlativas es verdadero?**).
- (VIII). Un programa que contenga propiedades anidadas en tres o más niveles de indentación sin perder el scope.
- (IX). Un programa que contenga comentarios entremezclados con definiciones estructurales.
- (X). Un programa que encadene el flujo completo: definición de una entidad, su regla de exención, y la regla transaccional correspondiente.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa malformado con errores de indentación (mezcla inconsistente de espacios/tabulaciones o pérdida de jerarquía en las viñetas).
- (II). Un programa que omita atributos obligatorios definidos en la estructura raíz (ej. declarar un proveedor sin **condicion_iva**).
- (III). Un programa que intente comparar tipos incompatibles en una regla de control (ej. comparar un **texto** de fecha con un valor numérico).
- (IV). Un programa que asigne dos entidades o reglas transaccionales con el mismo identificador exacto (colisión de nombres).
- (V). Un programa que utilice palabras reservadas del lenguaje (ej. **cliente**, **regla_pago**) como identificadores para nombres de entidades.

6. Ejemplos

A continuación, se presentan fragmentos de la sintaxis declarativa del lenguaje:

Ejemplo 1: Reglas de Facturación y Control de Exenciones (Ventas) Abstracción para la validación de certificados y la consulta a padrones impositivos al facturar a un cliente.

```
cliente "TechCorp S.R.L.":
  * tipo_persona: Juridica
  * condicion_iva: ResponsableInscripto
  * jurisdiccion_principal: CABA

regla_exencion "Control_Certificados":
  * evaluacion_al_facturar:
    - condicion_validez: fecha_factura menor o igual a fecha_hasta
    - aplicar_si_valido:
      * accion: anular_retencion_percepcion
      * aplicar_porcentaje: porcentaje_exencion
    - aplicar_si_vencido:
      * accion: ignorar_certificado

regla_facturacion "Venta_Servicios":
  * aplicar_sobre: "TechCorp S.R.L."
  * agregar_percepcion_iibb:
    - origen: "Padrón AGIP"
  * calcular_retencion_ganancias:
    - concepto: "Locación de Servicios"
    - consultar_escala: "AFIP_Resolucion_830"
```

Ejemplo 2: Auditoría Legal y Control de Proveedores (Compras) Abstracción de controles para pagos a regímenes simplificados, evitando multas fiscales y juicios laborales encubiertos.

```
regla_pago "Control_Proveedores_Monotributo":
  * aplicar_sobre_condicion: Monotributista

  * controlar_limites_operacion:
    - parametro: precio_unitario_facturado
    - validar_contra: "AFIP_Tope_Unitario_Bienes_Muebles"
    - accion_si_supera:
      * advertencia: "El proveedor supera el límite del régimen."
      * aplicar_retencion_ganancias: "Escala_No_Inscripto"

  * auditar_riesgo_laboral:
    - parametro: numero_factura
    - historial_requerido: ultimos_3_meses
    - condicion_alerta: facturas_correlativas es verdadero?
    - accion_si_cumple:
      * advertencia: "ALERTA LEGAL: Riesgo de relación de dependencia."
      * pausar_pago: verdadero
```

Ejemplo 3: Automatización Contable (Padrones y Cierres) Programación de tareas en segundo plano para sincronizar bases de datos y generar los artefactos requeridos para liquidar impuestos.

```
sincronizacion_padrones:
  * padron AGIP:
    - tipo: mensual
    - dia_descarga: 1
    - fuente: "agip.gob.ar/ws/padron_magnitudes"

cierre_mensual "Reportes_Liquidacion":
  * ejecucion: ultimo_dia_mes
  * enviar_a: "estudio_contable@empresa.com.ar"
  * generar_libro_iva:
    - modulo: Ventas
    - modulo: Compras
  * formatos_salida:
    - planilla_auditoria: Excel
    - importacion_arca: TXT_LibroIVADigital
    - importacion_nacional: TXT_SICORE
```